

EM-FDR-Bagging: Robust Estimation of the Connectivity Matrix from Multi-State Spiking Data

A bagged, scramble-calibrated wrapper around PRISM-EM

Jan Balewski, NERSC

June 16, 2026

Abstract

This document describes **EM-FDR-Bagging**, a statistical wrapper around the PRISM-EM fitter that aims to recover the shared connectivity matrix $A \in \mathbb{R}^{N \times N}$ of a non-stationary, multi-state Poisson GLM *robustly*, i.e. with explicit control of false edges and with per-edge uncertainty estimates, on real data where no ground truth exists. The method combines three ideas. First, *block bagging*: the recording is resampled into multiple overlapping bags built from spaced contiguous time blocks, and PRISM-EM is fit independently on each bag. Second, a *circular-shift null*: within each bag the fitted state sequence is frozen and the connectivity and state biases are re-estimated many times on per-neuron time-scrambled spikes, producing a null distribution of edge weights that have been passed through exactly the same estimation machinery as the real weights. Third, *per-row pooled significance with cross-bag stability selection*: edges are tested against a row-pooled null and retained only if they are selected reliably across bags. The procedure yields, for every surviving edge, a selection frequency, a bagged point estimate, a bootstrap SD, and a null-referenced effect size. We treat PRISM-EM as a black box exposed through two entry points and specify everything needed for an implementer who has the PRISM-EM code in hand.

Contents

1	Scope and Inputs	3
2	The PRISM-EM Workhorse and Its Two Entry Points	3
2.1	What PRISM-EM fits	3
2.2	Entry point 1: full fit	3
2.3	Entry point 2: M-step-only refit with locked state sequence	3
3	Method Overview	4
4	Stage (a.1): Block Bagging	5
5	Stage (a.2): Real Fit and the Circular-Shift Null	5
5.1	Real fit on the bag	5
5.2	The null: per-neuron independent circular shifts	6
5.3	Null re-estimation through the locked M-step	6
5.4	How many scrambles are needed	7
6	Stage (a.3): Per-Bag Outputs	7
7	Stage (b): Aggregation, Significance, and Pruning	8
7.1	Per-row pooled null and per-bag selection	8
7.2	Cross-bag stability selection	8

8	Per-Edge Uncertainty: Two Distinct Errors	9
8.1	Null spread and effect size	9
8.2	Estimation variability: the across-bag bootstrap SD	9
8.3	Reported per-edge record	10
9	Implementation and Runtime Arguments	10
9.1	<code>prism_EM_FDR_Bags_train3c.py</code> : Stage (a)	10
9.2	<code>prism_EM_FDR_Bags_agregate3c.py</code> : Stage (b)	13
10	End-to-End Summary	14

1 Scope and Inputs

The input is a single spike-train recording of N neurons over T discrete time bins of width Δt , in the format consumed by PRISM-EM (`spikesData/<dataName>.spikes.npz`). The target regime, which sizes every numerical choice below, is: $N = 50\text{--}500$ neurons; recordings of 10–60 minutes at $\Delta t = 10\text{ ms}$, so T ranges from 6×10^4 to 3.6×10^5 bins; per-neuron firing rates of 1–5 Hz; state-switching rates of 5–15 transitions per minute; and a small number of latent states, typically $M \approx 3$. Crucially, the method is designed for application to *real* data: it never references ground truth, and every quantity it reports is computed from the data alone.

The deliverable is not a single point estimate of A but a vetted edge list. For each off-diagonal entry (i, j) that survives the procedure we report a point estimate, a statistical uncertainty, a stability (selection) frequency, and an effect size relative to the null. Self-couplings A_{ii} are excluded from testing throughout.

2 The PRISM-EM Workhorse and Its Two Entry Points

EM-FDR-Bagging treats PRISM-EM as a callable. We summarize only what the wrapper relies on; the full model, objective, and defaults are documented separately in the PRISM-EM specification.

2.1 What PRISM-EM fits

PRISM-EM fits a non-stationary Poisson GLM in which all M latent states share one connectivity matrix A but carry state-specific bias vectors B_m . With c_t the simplex coefficients at bin t and $B_t = \sum_m c_{mt} B_m$, the rate is $\mu_t = \exp(\min(AY_{t-1} + B_t, \eta_{\text{clip}})) \Delta t$ and $Y_t \sim \text{Poisson}(\mu_t)$. Training is block coordinate descent: an E-step infers $c_{1:T}$ by simplex-projected gradient descent and Viterbi-decodes them to a hard one-hot state sequence $\hat{S}_{1:T}$; an M-step then updates $A, \mathbf{B}$ by Adam under the frozen one-hot assignment, with off-diagonal L_1 shrinkage (strength λ_3), a spectral-radius projection $\rho(A) \leq \rho_{\text{max}}$, and a learning-rate decay. The shrinkage, projection, and decay each switch on only after a configurable number of EM iterations. The fitted A is therefore the output of a specific, deterministic-in-configuration pipeline of regression plus shrinkage plus spectral projection — a fact the null construction below depends on.

2.2 Entry point 1: full fit

$$\text{full_fit}(\text{spikes}, M, \theta_{\text{EM}}) \longrightarrow (\hat{A}, \hat{S}_{1:T}, \hat{\mathbf{B}}, \text{history})$$

This is the ordinary PRISM-EM run: full E/M alternation on the supplied spikes, returning the fitted connectivity $\hat{A}$, the decoded one-hot state sequence $\hat{S}_{1:T}$, the per-state biases $\hat{\mathbf{B}}$, and the training history. Here θ_{EM} collects all PRISM-EM hyperparameters (number of EM iterations, M-step epochs, learning rates, $\lambda_2, \lambda_3, \rho_{\text{max}}$, the delay schedules, etc.).

2.3 Entry point 2: M-step-only refit with locked state sequence

$$\text{mstep_locked}(Y_{\text{prev}}, Y_{\text{curr}}, \hat{S}^{\text{lock}}, A^{\text{init}}, \hat{\mathbf{B}}^{\text{init}}, \theta_{\text{M}}) \longrightarrow (A, \mathbf{B})$$

This entry point runs *only* the M-step. It accepts spike pairs $(Y_{\text{prev}}, Y_{\text{curr}})$, an externally supplied and *frozen* state sequence $\hat{S}^{\text{lock}}$ (used directly as the one-hot assignment, with no E-step and no Viterbi re-decoding), initial parameters $(A^{\text{init}}, \hat{\mathbf{B}}^{\text{init}})$, and returns the re-estimated connectivity A and the refit state biases $\mathbf{B}$. In the null workflow the initial A and $\mathbf{B}$ are not copied from the real time-ordered fit. Instead, for every scramble they are initialized freshly from the scrambled data, or randomly, according to `null_A_init` and `null_B_init`. Re-fitting $\mathbf{B}$ in the scrambled context lets the null absorb state-specific baseline effects, so the test of A is not made artificially harsh by a bias mismatch; fresh A initialization avoids seeding the null from the real edge pattern.

Locked-state time indexing. For each lag-1 training pair (Y_{t-1}, Y_t) , the locked state used by the M-step is $\hat{S}_{t-1}^{\text{lock}}$, i.e. the state attached to the input/history bin. Equivalently, if the spike pairs have length $T - 1$, the state covariate rows are formed from $\hat{S}_{0:T-2}^{\text{lock}}$, converted to one-hot vectors. This convention is part of the model definition for lag-1 fitting and must be used identically for the real time-ordered fit and for every scrambled locked refit.

The locked M-step is configured to reproduce the real pipeline exactly: the same L_1 strength λ_3 , the same spectral projection ρ_{max} , and the same learning-rate decay as the corresponding `full_fit`. Because there is no E-step here, the EM iteration counters that gate PRISM-EM’s delayed schedules are not meaningful; instead all schedule-gated operations (shrinkage, spectral projection, decay) are **active from the first M-step epoch**. The rationale is that the null estimate must traverse the same shrinkage and constraint geometry as the converged real estimate, which lives in the late-EM regime where all schedules have already fired.

Implementation note. In the current `ver3c` code these two entry points live in `PrismEM_Workhorse3c.py`: `run_full_fit` and `run_locked_mstep`. The locked M-step consumes externally supplied spike pairs, an externally supplied $\hat{S}^{\text{lock}}$, and explicit initial A and B . Its lag-1 state alignment is $\hat{S}_{t-1}^{\text{lock}}$ for pair (Y_{t-1}, Y_t) , implemented by `onehot_from_prev_states`.

3 Method Overview

EM-FDR-Bagging has two stages, and the document is organized to match the intended software structure.

Stage (a), per-bag fitting (Sections 4–6). For a single bag: assemble a resampled time series from spaced blocks, run `full_fit` to obtain the real $(\hat{A}^{(b)}, \hat{S}^{(b)})$, then run P scramble refits through `mstep.locked` with the state sequence frozen, obtaining a set of null connectivity matrices. Write everything to disk. This stage is one self-contained program; it is run N_{bag} times, once per bag, as independent jobs.

Stage (b), aggregation and pruning (Section 7). Read all per-bag outputs from a directory, build per-row pooled null distributions, perform per-bag selection and cross-bag stability selection, and emit the final vetted edge list with uncertainties.

The symbols introduced for the wrapper, kept distinct from PRISM-EM’s N, M, T , are collected in Table 1.

Symbol	Meaning	Nominal value
N_{blk}	blocks per bag	10
L_{blk}	block length (s)	60
$L_{\text{blk}}^{\text{bin}}$	block length (bins) = $L_{\text{blk}}/\Delta t$	6000
R_{time}	time range for block starts (s)	e.g. [1000, 1600]
N_{bag}	number of bags	5–10
P	scrambles per bag	4–8
$\delta_{\text{blk}}^{\text{min}}$	min separation between block starts (s)	1
$\delta_{\text{roll}}^{\text{min}}$	roll exclusion-zone half-width (s)	2
q	per-bag per-row selection quantile level	e.g. 0.99
π_{thr}	cross-bag selection-frequency threshold	0.6–0.8
M, N, T	states, neurons, bins (PRISM-EM)	≈ 3 ; 50–500; 6×10^4 – 3.6×10^5

Table 1: Symbols introduced by EM-FDR-Bagging and their nominal values. Every numeric choice in the text is written as “symbol = value” so it can be adjusted.

4 Stage (a.1): Block Bagging

A bag is a surrogate recording assembled by concatenating $N_{\text{blk}} = 10$ contiguous blocks, each of length $L_{\text{blk}} = 60\text{ s}$ (i.e. $L_{\text{blk}}^{\text{bin}} = 6000$ bins at $\Delta t = 10\text{ ms}$), drawn from a designated time range R_{time} of the original recording. The reassembled bag has length $N_{\text{blk}} \times L_{\text{blk}}^{\text{bin}}$ bins (here 6×10^4 bins, i.e. 10 minutes), which is the sequence PRISM-EM fits.

Block placement. Block start positions are bin indices drawn from the time range. The range R_{time} is given in seconds and must leave room for a full block, so admissible starts (in bins) lie in $[R_{\text{time}}^{\text{start}}/\Delta t, R_{\text{time}}^{\text{end}}/\Delta t - L_{\text{blk}}^{\text{bin}}]$. Within that range a start may fall on essentially any bin, so the number of candidate starts is large (tens of thousands), and exact collisions between two chosen starts are vanishingly unlikely. The genuine hazard is not identical starts but *near-identical, heavily overlapping* blocks: two starts one bin apart share all but one bin of content and would inject a near-duplicate stretch into the bag. We therefore impose a minimum separation $\delta_{\text{blk}}^{\text{min}} = 1\text{ s}$ (100 bins) between any two chosen starts and sample by rejection: draw N_{blk} starts uniformly from the admissible range, reject and redraw any draw that violates the minimum-separation constraint, and try up to `max.block.draw.trials`=30 times for each new block. If all 30 attempts fail, accept the 31st draw regardless of the separation constraint and mark the violation in the job metadata. This bounded retry policy avoids silent infinite loops while still giving the ordinary case many chances to avoid near-duplicates.

Reference chronological bag. The special job `bag_idx=0` is not randomly resampled. Its block starts follow the input-data timeline: block 0 begins at the start of `--time.range.sec`, block 1 begins one block length later, and so on. This bag is used as the Stage (b) reference for time-series fields, so state-assignment plots from the aggregate output remain aligned to the original simulation timeline when ground truth is available. All `bag_idx>0` jobs use the random block-bootstrap sampler described above.

Blocks are non-circular. Each block is a literal contiguous slice of the recording, clipped at the window end; no wrap-around is used in block extraction. Lag structure within a block is preserved exactly, so the first lag pair (Y_0, Y_1) at a block boundary is the only mild discontinuity, and with $L_{\text{blk}}^{\text{bin}} = 6000$ bins per block the boundary fraction is negligible. By default these cross-block boundary pairs are included rather than masked, because the assembled bag is passed to `full_fit` as an ordinary spike sequence and the number of such pairs is only $N_{\text{blk}} - 1$ out of $T_b - 1$. The realized block start offsets and the assembled-sequence boundary pair indices are stored in the per-bag job record so their influence can be audited, or a stricter masked-pair variant can be evaluated later.

Why overlapping bags. Bags are deliberately allowed to overlap in source time across the N_{bag} runs; this is a block bootstrap on a single finite recording. Overlap induces positive correlation between bags, which has consequences for the uncertainty estimates (Section 8) and the stability bound (Section 7); these are stated honestly where they bite. The overlap can be reduced — and its impact studied — by lengthening the total recording while holding N_{blk} small, which the parameter layout supports directly.

5 Stage (a.2): Real Fit and the Circular-Shift Null

5.1 Real fit on the bag

For bag b , call entry point 1 on the assembled bag:

$$(\hat{A}^{(b)}, \hat{S}_{1:T_b}^{(b)}, \hat{\mathbf{B}}^{(b)}) = \text{full_fit}(\text{bag}_b, M, \theta_{\text{EM}}), \quad T_b = N_{\text{blk}} L_{\text{blk}}^{\text{bin}}.$$

This $\hat{A}^{(b)}$ is the bag’s real connectivity estimate; the frozen state sequence $\hat{S}^{(b)}$ is reused unchanged for all scrambles below.

5.2 The null: per-neuron independent circular shifts

The null hypothesis for an edge is “no directed lag-1 coupling from neuron j to neuron i beyond what the bias/state structure already explains.” We realize it by destroying cross-neuron temporal alignment while preserving each neuron’s own marginal statistics and the state timeline. Concretely, for scramble $p = 1, \dots, P$ we draw an independent integer circular shift $\delta_n^{(p)}$ for each neuron n and roll that neuron’s spike train:

$$\tilde{Y}_{t,n}^{(p)} = Y_{(t-\delta_n^{(p)}) \bmod T_b, n}.$$

Because every neuron is shifted by a different random amount, the precise lag-1 relationship that A captures is broken, while each neuron’s firing rate, burstiness, and count distribution are untouched (a circular roll is a pure permutation of a neuron’s own time axis). The state sequence $\hat{S}^{(b)}$ is *not* rolled: it is held fixed so that the null asks specifically about coupling, not about state structure.

Two-sided exclusion zone on each shift. A shift near 0 leaves the data almost unshifted and a shift near T_b wraps back to almost unshifted; both leave the real lag structure largely intact and would contaminate the null toward the alternative. We therefore restrict each shift to

$$\delta_n^{(p)} \in [\delta_{\text{roll}}^{\text{min}, \text{bin}}, T_b - \delta_{\text{roll}}^{\text{min}, \text{bin}}], \quad \delta_{\text{roll}}^{\text{min}, \text{bin}} = \delta_{\text{roll}}^{\text{min}} / \Delta t,$$

with $\delta_{\text{roll}}^{\text{min}} = 2\text{ s}$ (200 bins). The exclusion is two-sided by construction: both the small-shift and the near-full-wrap small-effective-shift regions are forbidden.

No pairwise separation among neurons. The shifts of different neurons need not be separated from one another. The null only requires that each neuron’s shift is individually non-trivial (the two-sided zone above) and that shifts are drawn independently across neurons; a coincidence in which two neurons happen to share the same shift is harmless to null validity. This matters in practice: with a 10-minute bag and a 2 s separation there would be only ~ 300 “slots,” yet a bag may contain up to 500 neurons, so any attempt to enforce pairwise distinct shifts would be infeasible by pigeonhole. We deliberately avoid that requirement. The implemented draw routine simply samples independently from the allowed two-sided interval for each neuron.

5.3 Null re-estimation through the locked M-step

For each scramble p we recompute connectivity with the state sequence frozen, via entry point 2:

$$(A^{\text{null}, (b,p)}, \mathbf{B}^{\text{null}, (b,p)}) = \text{mstep_locked}(\tilde{Y}_{\text{prev}}^{(p)}, \tilde{Y}_{\text{curr}}^{(p)}, \hat{S}^{(b)}, A^{\text{init}, (b,p)}, \mathbf{B}^{\text{init}, (b,p)}, \theta_M).$$

The locked state attached to pair $(\tilde{Y}_{t-1}^{(p)}, \tilde{Y}_t^{(p)})$ is $\hat{S}_{t-1}^{(b)}$. The state sequence itself remains the decoded sequence from the real, time-ordered bag fit; only the spike counts entering the null M-step are circularly shifted. The null initialization is fresh for each scramble: $A^{\text{init}, (b,p)}$ is either computed from the scrambled spike train (`null_A_init=scrambled_data`, the default) or drawn randomly (`null_A_init=rand`), and $\mathbf{B}^{\text{init}, (b,p)}$ is likewise initialized from scrambled data (`null_B_init=scrambled_data`, the default) or randomly (`null_B_init=rand`). The real time-ordered $\hat{A}^{(b)}$ is deliberately not used to seed the null refit. The configuration θ_M matches the real pipeline exactly (Section 2.3): identical λ_3 , identical ρ_{max} , identical learning-rate decay, all active from the first epoch. This is the crux of the calibration: the real $\hat{A}^{(b)}$ and the null $A^{\text{null}, (b,p)}$ are both outputs of the *same* regression, shrinkage, and spectral projection, applied to the same state timeline, differing only in whether cross-neuron alignment was destroyed and in the resulting refit biases. A significance statement of the form “this edge is larger than the same machinery produces on scrambled data” is therefore meaningful; using a raw, unshrunk M-step for the null would compare against a differently biased quantity and invalidate the test.

5.4 How many scrambles are needed

A useful efficiency follows from testing significance per row rather than per edge (Section 7). For row i we pool the $(N - 1)$ off-diagonal null weights of that row across all scrambles. A *single* scramble therefore already contributes $\sim (N - 1)$ null samples to row i 's distribution — of order 199 for $N = 200$. With a modest $P = 4\text{--}8$ the pooled per-row null holds $(N - 1) \times P \approx 800\text{--}1600$ samples, ample to resolve a 1% tail. By contrast a *per-edge* null would need on the order of hundreds of scrambles just to populate a single edge's distribution, which is infeasible at this budget. The two regimes are not equivalent: the $(N - 1)$ samples within one scramble are not independent (they share a single scrambled realization of the data), so increasing P still helps by adding genuinely independent null realizations and by averaging out scramble-to-scramble variability. The recommendation is to keep P small (4–8) but greater than one, precisely because per-row pooling supplies the raw sample count while extra scrambles supply independence.

6 Stage (a.3): Per-Bag Outputs

Each per-bag job writes a self-contained file to a shared results directory, with a name carrying the bag index so that N_{bag} parallel jobs do not collide. The file contains, at minimum: the real connectivity $\hat{A}^{(b)} \in \mathbb{R}^{N \times N}$; the stack of null connectivities $\{A^{\text{null},(b,p)}\}_{p=1}^P$ of shape (P, N, N) ; the frozen state sequence $\hat{S}^{(b)}$; the realized block start indices and the assembled-sequence boundary pair indices; any block-start constraint violations caused by exhausting `max_block_draw_trials`; the realized per-neuron shifts (for reproducibility and for the data-reuse study); the stack of refit null biases $\{\mathbf{B}^{\text{null},(b,p)}\}_{p=1}^P$ of shape (P, M, N) ; the fitted off-diagonal sparsity of $\hat{A}^{(b)}$ (number and fraction of surviving edges) as a sanity cross-check; and the PRISM-EM training-history summary (final objective, iteration count, whether each delayed schedule fired, final state occupancy). The assembled bag spike train itself is not saved, because the ordered block starts are sufficient to reconstruct it from the original input spikes if needed. Stage (b) consumes only these files and never re-runs PRISM-EM.

Per-bag program output naming. One invocation of `prism_EM_FDR_Bags_train3c.py` produces one bag file containing the real time-ordered fit and all P scramble refits for that bag. The filename is deterministic and includes `dataName`, `bagsTag`, and `bag_idx`, for example `<dataName>_<bagsTag>.bag003.prismFDRbag.npy`. The real fit fields are written in the same schema as the standalone `prism_EM_train3c.py` output, so the usual baseline fit evaluation can be run on the real part of a bag without a separate training job.

Per-bag driver runtime controls. The bag driver inherits the ordinary PRISM-EM training controls (`num_states`, `num_em_iters`, `m_epochs`, learning rates, $\lambda_2, \lambda_3, \rho_{\text{max}}$, schedule delays, batch size, and initialization modes for the real fit). Its additional controls are: `bag_idx`, `bagsTag`, `time_range_sec`, `num_blocks`, `block_len_sec`, `min_block_start_sep_sec`, `max_block_draw_trials`, `num_scrambles`, `min_roll_shift_s`, `null_A_init`, and `null_B_init`. There is no separate `null_m_epochs` parameter: the locked null M-step always inherits the M-step epoch count and optimizer configuration from the real time-ordered fit, with pruning, spectral projection, and learning-rate decay active from the first locked epoch. There is also no option to save the assembled bag spike train; the ordered block starts are the reproducible representation of the bag.

Compute layout. Each per-bag job is a single SLURM job on a single node using $4 \times \text{A100}$, matching the existing PRISM-EM launch (`torchrun --standalone --nnodes=1 --nproc_per_node=4`). The P scramble refits reuse the same node and the frozen $\hat{S}^{(b)}$, so they are cheap relative to the full fit. The N_{bag} jobs are mutually independent and submitted in parallel.

7 Stage (b): Aggregation, Significance, and Pruning

Stage (b) is a single postprocessing program pointed at the results directory. It performs per-row null pooling, per-bag selection, and cross-bag stability selection, then reports the vetted edge list with uncertainties.

7.1 Per-row pooled null and per-bag selection

We test significance *per row*, exploiting the assumption that the incoming edges to a given target neuron i are exchangeable under the null: they share that target neuron’s firing statistics, state occupancy, and observation scale. This is a much milder assumption than per-edge normality. The row convention is A_{ij} = effect of source neuron j on target neuron i ; therefore this row-pooled test calibrates the incoming weights to one target neuron. The separate biological interpretation of a source neuron as broadly excitatory or inhibitory is read down a column of retained edges, across the downstream targets it controls. The per-row exchangeability assumption is what makes the small scramble budget viable (Section 5.4).

For bag b and row i , pool the off-diagonal null weights across all scrambles of that bag,

$$\mathcal{N}_i^{(b)} = \{ A_{ij}^{\text{null},(b,p)} : j \neq i, p = 1, \dots, P \}, \quad |\mathcal{N}_i^{(b)}| = (N - 1) P,$$

and form a per-row threshold as the upper- q quantile of the magnitudes,

$$\tau_i^{(b)} = \text{Quantile}_q(\{|x| : x \in \mathcal{N}_i^{(b)}\}), \quad q = 0.99 \text{ (nominal)}.$$

An edge is *selected in bag b* if its real magnitude exceeds the row threshold,

$$\text{sel}_{ij}^{(b)} = \mathbb{K}[|\hat{A}_{ij}^{(b)}| > \tau_i^{(b)}], \quad i \neq j.$$

Equivalently, each real edge can be converted to a within-row p -value against the pooled null and the Benjamini–Hochberg procedure applied within the row; the quantile-threshold form above is the operational default. The expected number of edges selected per bag and per row is recorded as a diagnostic (it is the quantity q_Λ of the stability bound below); it is *measured*, never prescribed, so the algorithm discovers how many edges it keeps rather than being told a target sparsity.

7.2 Cross-bag stability selection

A single bag’s selection is noisy. We retain only edges that are selected reliably across bags. Define the selection frequency

$$\pi_{ij} = \frac{1}{N_{\text{bag}}} \sum_{b=1}^{N_{\text{bag}}} \text{sel}_{ij}^{(b)},$$

and keep edge (i, j) in the final network if $\pi_{ij} \geq \pi_{\text{thr}}$, with $\pi_{\text{thr}} = 0.6$ – 0.8 (nominal). This is Meinshausen–Bühlmann stability selection: the data are perturbed by resampling, the base selector is run on each perturbation, and only consistently chosen variables survive. For example, the implementation default `--sel_prob 0.7` means that an off-diagonal edge must be selected in at least 70% of the bags to survive. The value assigned to a surviving edge is not weighted by the selection frequency or by any bag quality score; it is the plain arithmetic mean of $\hat{A}_{ij}^{(b)}$ over the bags in which that edge passed the per-bag null threshold. Bags where the edge did not pass the per-bag selector do not enter this final `A_hat` value, although the all-bag mean is also saved as a diagnostic.

Reported stability bound. The implementation exposes the pair (q, π_{thr}) : a per-bag per-row quantile level q and a cross-bag frequency threshold π_{thr} . It also reports the Meinshausen–Bühlmann-style stability-selection diagnostic

$$\mathbb{E}[V] \leq \frac{1}{2\pi_{\text{thr}} - 1} \cdot \frac{q_{\Lambda}^2}{p_{\text{cand}}},$$

where $\mathbb{E}[V]$ is the expected number of false edges, q_{Λ} is the *measured* average number of edges selected per bag, and $p_{\text{cand}} = N(N - 1)$ is the number of candidate off-diagonal edges. The current program does not derive π_{thr} from a target false-edge count; the user sets `--per_bag_quantile` and `--sel_prob` directly, and the bound is saved as a diagnostic. The honest caveat is that the bound assumes independent exchangeable bags, whereas overlapping-block bags are positively correlated, so the reported value should be read as an approximate stability diagnostic rather than as a guaranteed FDR.

8 Per-Edge Uncertainty: Two Distinct Errors

The design furnishes two genuinely different per-edge “errors,” which answer different questions and must not be conflated.

8.1 Null spread and effect size

For row i , the pooled null $\mathcal{N}_i = \bigcup_b \mathcal{N}_i^{(b)}$ has a mean and standard deviation,

$$m_i^{\text{null}} = \text{mean}(\mathcal{N}_i), \quad s_i^{\text{null}} = \text{SD}(\mathcal{N}_i),$$

which characterize the *noise floor*: how large an edge weight looks purely by chance once cross-neuron alignment is destroyed and the same machinery is applied. This is not the uncertainty of the estimate; it is the spread of the null. It does, however, define a natural null-referenced effect size,

$$z_{ij} = \frac{\bar{A}_{ij} - m_i^{\text{null}}}{s_i^{\text{null}}},$$

a per-edge signal-to-noise ratio against the scramble null, where $\bar{A}_{ij}$ is the bagged point estimate defined next.

8.2 Estimation variability: the across-bag bootstrap SD

The quantity one usually means by “the statistical error of an edge” is the sampling variability of its fitted weight, and bagging estimates it directly. Each bag yields a real fitted weight $\hat{A}_{ij}^{(b)}$ on a different resampled stretch of data; their spread across bags is a block-bootstrap estimate of the estimator’s sampling distribution. We report the bagged mean and bootstrap SD,

$$\bar{A}_{ij} = \frac{1}{B_{ij}} \sum_{b \in \mathcal{B}_{ij}} \hat{A}_{ij}^{(b)}, \quad \widehat{\text{SD}}_{ij} = \sqrt{\frac{1}{B_{ij} - 1} \sum_{b \in \mathcal{B}_{ij}} (\hat{A}_{ij}^{(b)} - \bar{A}_{ij})^2},$$

where $\mathcal{B}_{ij}$ is the set of bags entering the average and $B_{ij} = |\mathcal{B}_{ij}|$.

Which bags to average over. A bag in which edge (i, j) was pruned to exactly zero by L_1 contributes a structural zero, not a noisy nonzero. Two summaries are therefore reported. The *selection-conditional* estimate averages only over bags where the edge survived ($\mathcal{B}_{ij} = \{b : \text{sel}_{ij}^{(b)} = 1\}$); it reflects the typical magnitude when the edge is detected but is biased upward in magnitude. The *all-bag* estimate averages over every bag ($\mathcal{B}_{ij} = \{1, \dots, N_{\text{bag}}\}$, structural zeros included); it is diluted toward zero in proportion to how often the edge is absent. We recommend reporting the selection-conditional mean and bootstrap SD *together with* the selection frequency π_{ij} , so that the dilution is visible rather than hidden: π_{ij} and $\bar{A}_{ij}$ together convey both how often and how strongly an edge appears.

Honest caveat on calibration. The bags are overlapping block-bootstrap samples from one finite recording, not independent datasets. Overlap makes bags more alike than independent draws, so the naive across-bag SD *underestimates* the true sampling error; with only $N_{\text{bag}} = 5\text{--}10$ bags it is additionally coarse (few degrees of freedom). Accordingly $\widehat{\text{SD}}_{ij}$ should be used as a *relative* reliability measure — excellent for ranking edges and for flagging unstable ones — rather than as a calibrated frequentist standard error supporting a strict confidence interval. The degree of underestimation can be probed empirically by lengthening the recording and shrinking the block overlap (Section 4), which is a deliberate affordance of the design.

8.3 Reported per-edge record

For every edge that passes stability selection ($\pi_{ij} \geq \pi_{\text{thr}}$) the pipeline emits

$$(i, j, \pi_{ij}, \bar{A}_{ij}, \widehat{\text{SD}}_{ij}, z_{ij}),$$

together with the all-bag mean for comparison and the row null summary ($m_i^{\text{null}}, s_i^{\text{null}}$). The selection frequency π_{ij} , the bagged point estimate, the bootstrap SD, and the null-referenced effect size are complementary: an edge that is frequently selected, large relative to its bootstrap SD, and far into the null tail is trustworthy; an edge strong in one bag but rarely selected, or with bootstrap SD comparable to its magnitude, is flagged as unreliable.

9 Implementation and Runtime Arguments

This section records the concrete command-line interface used by the current implementation. Stage (a) produces one bag file per invocation. Stage (b) reads the ordered set of bag files and writes a single aggregate `.prismEM.npz` file that can be opened by the existing `prismEM.eval3c.py` machinery.

9.1 `prismEM.FDR.Bags.train3c.py`: Stage (a)

This program is run once per bag, normally as one multi-GPU job:

```
torchrun --standalone --nnodes=1 --nproc_per_node=4 ./prismEM.FDR.Bags.train3c.py ...
```

It assembles one block-resampled bag, performs the real time-ordered PRISM-EM fit on that bag, runs the requested circular-shift locked-M-step null refits, and writes one self-contained bag file.

Input and output naming. The input spike file is

`<basePath>/spikesData/<dataName>.spikes.npz.`

The output directory is `fdr_out_dir` if supplied, otherwise

`<basePath>/prismFDR.`

The output file is deterministic in `dataName`, `bagsTag`, and `bag_idx`:

`<basePath>/prismFDR/<dataName>.<bagsTag>.bag003.prismFDRbag.npz`

for `bag_idx = 3` and default output directory. The bag index is not only a label: it also enters the random seed stream, so rerunning the same command with the same `bag_idx` reproduces the same bag and scramble shifts, while changing only `bag_idx` produces a different bag file. The one exception is `bag_idx=0`: its block starts are chronological by construction, while its scramble shifts still use the bag-indexed random seed.

A single Stage (a) bag can be evaluated directly by passing the bag stem to `prismEM.eval3c.py`; if `dataName` contains a `bagNNN` token, the evaluator reads `<basePath>/prismFDR/<dataName>.prismFDRbag.npz` instead of looking under `prismFit`.

Required arguments.

- `--basePath` Root directory containing `spikesData/` and receiving `prismFDR/`. This is the same run directory used by the plain PRISM-EM trainer.
- `--dataName` Short dataset name. The program reads `<dataName>.spikes.npz` from `spikesData/`.
- `--num_states` Number M of latent states in the switching Poisson GLM.
- `--bag_idx` Integer bag index. It is used in the output filename and in the per-bag RNG seed. For `bag_idx=0`, the bag blocks are chronological, starting at the time-range start and advancing by one block length per block; other bag indices draw random block starts.
- `--time_range_sec` `START END` Time range, in seconds, from which block starts are drawn.

Bagging and scramble arguments.

- `--fdr_out_dir` Optional output directory for Stage (a) bag files. Default: `<basePath>/prismFDR`.
- `--bagsTag` `TAG` Optional alphanumeric tag appended after `dataName` in Stage (a) output names. If omitted or set to `None`, the program derives a deterministic four-character alphanumeric tag from the shared bagging/training configuration, excluding `bag_idx`.
- `--num_blocks` Number of contiguous blocks concatenated into one bag. Default: 10.
- `--block_len_sec` Length of each contiguous block in seconds. Default: 60.
- `--min_block_start_sep_sec` Desired minimum separation between block starts. Default: 1 second.
- `--max_block_draw_trials` Number of rejection-sampling attempts used to satisfy the block-start separation constraint. If no candidate passes after this many attempts, the next draw is accepted and the violation is recorded in the output. Default: 30.
- `--num_scrambles` Number P of circular-shift null refits saved in this bag file. Default: 4.
- `--min_roll_shift_sec` Two-sided exclusion zone around zero circular shift for each neuron. A shift is drawn from the interval excluding shifts smaller than this value and shifts closer than this value to a full wrap. Default: 2 seconds.
- `--null_A_init` Fresh A initialization mode for every null refit. Values: `scrambled.data` (default) or `rand`. The real-time $\hat{A}$ is deliberately not used to seed the null.
- `--null_B_init` Fresh B initialization mode for every null refit. Values: `scrambled.data` (default) or `rand`. The null B is refit and saved as `B_null`.

Inherited EM and M-step arguments. The Stage (a) driver shares the ordinary PRISM-EM workhorse with `prism_EM_train3c.py`.

- `--num_em_iters` Number of outer EM iterations for the real time-ordered bag fit. Default: 20.
- `--m_epochs` M-step Adam epochs per EM iteration. Default: 2. The null locked M-step uses the total count `num_em_iters × m_epochs`; there is no separate `null_m_epochs`.
- `--pgd_iter` PGD iterations per time bin in the E-step. Default: 5.
- `--lr_estep` PGD learning rate in the E-step. Default: 0.03.

`--lambda2` Temporal smoothness weight for the state probabilities. Default: 2.0.
`--lr_mstep` Adam learning rate for the M-step. Default: 0.003.
`--lr_end_factor` Final learning-rate multiplier after linear decay. Default: 0.1.
`--lambda3` L1 penalty weight on off-diagonal A . Default: 0.02.
`--rho_max` Target spectral radius for soft correction of A . Default: 0.95.
`--prescale_m_step_4_ArhoMax` M-step batch interval for spectral-radius projection checks. Default: 120.
`--delay_em_iter_4_ArhoMax` First real-fit EM iteration after which $\rho_{\max}$ correction is active. Default: 3.
`--delay_em_iter_4_lrDecay` First real-fit EM iteration after which learning-rate decay advances. Default: 1.
`--delay_em_iter_4_Aprune` First real-fit EM iteration after which L1 proximal pruning is active. Default: 1.
`--batch_size` M-step minibatch size. Default: 2048.
`--init_states` Initial latent-state mode for the real fit: `data` (default) or `rand`.
`--init_A` Initial A mode for the real fit: `data` (default) or `rand`.
`--init_B` Initial B mode for the real fit: `data` (default) or `rand`.
`--init_A_Tmax` Maximum number of bins used for data-driven A initialization. Default: 50000.
`--decode_dwell_sec` Dwell-time prior used in Viterbi decoding. Default: 0.3 seconds.
`--seed` Base random seed. Default: 42; the effective seed is offset by `bag_idx`.
`--fitName` Accepted by the shared parser for ordinary EM compatibility, but Stage (a) output naming is determined by `dataName` and `bag_idx`.
`--verb` Verbosity level. With the default verbosity, rank 0 reports runtime device information, real-fit elapsed time, and elapsed time for each scramble refit.

Saved Stage (a) contents. The file has the normal real-fit fields (`A_hat`, `B_hat`, `S_hat`, `c_hat`, training histories, etc.) plus `A_null`, `B_null`, `roll_shifts_bin`, `block_start_bins`, `block_start_sec`, `boundary_pair_indices`, `block_retry_violations`, and locked null M-step histories. The assembled bag spike train is not saved. Stage (a) metadata are grouped under `bagsFDR_stageA`. This includes the bag assembly controls, the circular-shift null-refit controls and per-scramble initialization diagnostics, and a `real_fit` sub-dict holding the usual PRISM-EM `train`, `init_A`, `init_B`, `init_state`, and `states_recovery_eval` metadata for the real time-ordered fit on that bag.

big_fit_bags.sh launch pattern. The helper script `big_fit_bags.sh` is a concrete multi-bag launch example for one dataset. It sets

```
basePath=/pscratch/sd/b/balewski/2026_causalNet_exp_ver3c
```

and a short input spike name such as `daleN100_2ba29b_c47b43`. It then defines

```
bagTag="j${SLURM_JOB_ID: -4}",
```

so all bags launched inside one SLURM job share the same Stage (a) extended name

`<shortN>_<bagTag>.`

For each requested bag index, the script runs `torchrun --standalone --nnodes=1 --nproc_per_node=4` on `prism_EM_FDR_Bags_train3c.py`. With the current loop for `bag` in 3 4 5, the expected output files are

```
<basePath>/prismFDR/<shortN>_<bagTag>.bag003.prismFDRbag.npz
<basePath>/prismFDR/<shortN>_<bagTag>.bag004.prismFDRbag.npz
<basePath>/prismFDR/<shortN>_<bagTag>.bag005.prismFDRbag.npz
```

If the commented loop for `bag` in 0 1 2 is used instead, the files are `bag000`, `bag001`, and `bag002`; `bag000` is the chronological reference bag. To aggregate the full set, Stage (b) must be called with `--dataName <shortN>_<bagTag>` and `--num_bags` equal to the number of contiguous bags starting from `bag000` that are present on disk.

9.2 prism_EM_FDR_Bags_aggregate3c.py: Stage (b)

This program is a single CPU postprocessing step. It reads the Stage (a) files, performs the per-row null thresholding and cross-bag stability selection, and writes one aggregate file in the same outer format expected by `prism_EM_eval3c.py`. The filename contains `aggregate` with one “g” because that is the implemented script name.

Input and output naming. The default input directory is

`<basePath>/prismFDR.`

For `num_bags = B`, the program expects the contiguous file set

```
<dataName>.bag000.prismFDRbag.npz
<dataName>.bag001.prismFDRbag.npz
...
<dataName>.bag{num_bags-1, zero padded}.prismFDRbag.npz
```

Here `dataName` is the extended Stage (a) name, i.e. `<originalDataName>_<bagsTag>`. The output stem is

`<dataName>_bags<num_bags>.`

and the output file is written to

`<basePath>/prismFit/<dataName>_bags<num_bags>.prismEM.npz.`

For example, with `dataName=daleN100_2ba29b_c47b43` and `num_bags=5`, the output stem is

`daleN100_2ba29b_c47b43_bags5.`

It can then be opened by

```
./prism_EM_eval3c.py \
  --basePath <basePath> \
  --dataName <dataName>_bags5 \
  -p a e f g
```

Runtime arguments.

- `--basePath` Root run directory. The default input is `<basePath>/prismFDR`; the aggregate output is written to `<basePath>/prismFit`.
- `--dataName` Extended Stage (a) output data name, normally `<originalDataName>_<bagsTag>`. This exact string is the prefix of all bag files consumed by Stage (b), e.g. `<dataName>.bag000.prismFDRbag.npz`.
- `--num_bags` Number of Stage (a) bag files to read. The implementation expects bags numbered contiguously from `bag000`.
- `--per_bag_quantile` Quantile q used for the per-row null magnitude threshold $\tau_i^{(b)}$. Default: 0.99.
- `--sel_prob` Cross-bag selection-frequency threshold π_{thr} . Default: 0.7.
- `--fdr_out_dir` Optional override for the Stage (a) bag directory. Default: `<basePath>/prismFDR`.
- `--verb` Verbosity level. Default: 1.

Aggregate output schema. The output `A_hat` is the final stable matrix: off-diagonal entries are the selection-conditional bagged means for edges with $\pi_{ij} \geq \text{sel_prob}$, and zero otherwise. With the default `--sel_prob 0.7`, this means retaining only edges present in at least 70% of bags; the retained edge value is the unweighted mean over only those bag instances where the edge was selected. Diagonal entries are the all-bag mean diagonals. `A_prune`, `neuron_type`, and `neuron_Sedge` are recomputed from this aggregate `A_hat` using the same source-column Dale convention as the ordinary EM output: `neuron_Sedge` is the off-diagonal column sum for each source neuron, `neuron_type` is inferred from that source-column sum, and `A_prune` keeps signs by source columns. The output `B_hat` is the mean across bags after aligning state rows to bag 0 by the fitted `B` vectors. The time-series fields `S_hat`, `c_hat`, `S_hat_CL`, and the EM histories are copied from reference bag 0 to preserve compatibility with the existing plotting code; those plots should therefore be interpreted as reference-bag diagnostics, while the connectivity fields are Stage (b) aggregates. Because `bag000` is chronological, these copied state assignments are aligned to the original input timeline for simulation truth comparisons. Stage (b) metadata are grouped under `bagsFDR_stageB`; Stage (a) metadata copied from reference bag 0 remain under `bagsFDR_stageA`.

Additional Stage (b) arrays include `A_bag`, `B_bag_aligned`, `state_permutation_to_ref`, `selected_mask`, `selected_in_bag`, `selection_count`, `selection_frequency`, `A_mean_selected`, `A_sd_selected`, `A_mean_all`, `A_sd_all`, `row_null_tau_bag`, `row_null_tau_mean`, `row_null_mean`, `row_null_sd`, `z_null`, and a compact selected-edge table with `edge_i`, `edge_j`, `edge_sel_freq`, `edge_A_mean`, `edge_A_sd_boot`, and `edge_z_null`.

10 End-to-End Summary

Stage (a), run N_{bag} times in parallel (one node, $4 \times \text{A100}$ each).

1. Assemble bag b : draw $N_{\text{blk}} = 10$ block starts in the time range R_{time} , enforce $\delta_{\text{blk}}^{\min} = 1$ s separation by rejection sampling (up to `max_block_draw_trials`=30 attempts per new block, then accept the next draw and record any violation), extract non-circular blocks of $L_{\text{blk}} = 60$ s, concatenate, and record the block offsets and boundary pair indices.
2. `full_fit` $\rightarrow (\hat{A}^{(b)}, \hat{S}^{(b)}, \hat{B}^{(b)})$.
3. For $p = 1, \dots, P$: draw independent per-neuron circular shifts in the two-sided zone $[\delta_{\text{roll}}^{\min, \text{bin}}, T_b - \delta_{\text{roll}}^{\min, \text{bin}}]$, roll spikes, and `mstep_locked` $\rightarrow (A^{\text{null}, (b, p)}, B^{\text{null}, (b, p)})$ with the real pipeline's $\lambda_3, \rho_{\text{max}}$, decay active from epoch one, $\hat{S}^{(b)}$ frozen and aligned as $\hat{S}_{t-1}^{(b)}$ for pair (Y_{t-1}, Y_t) , fresh A initialization controlled by `null_A_init` $\in \{\text{scrambled_data}, \text{rand}\}$, and fresh B initialization controlled by

`null_B_init` $\in \{\text{scrambled_data}, \text{rand}\}$. The number of locked M-step epochs and all other M-step hyperparameters are inherited from the real time-ordered fit configuration.

4. Write one file for bag b , named from `dataName` and `bag_idx`, containing $\hat{A}^{(b)}$, $\{A^{\text{null},(b,p)}\}$, $\{\mathbf{B}^{\text{null},(b,p)}\}$, $\hat{S}^{(b)}$, realized block starts, boundary pair indices, block-start retry violations, realized shifts, fitted sparsity, and history summary. Do not save the assembled bag spike train.

Stage (b), single postprocessing job.

1. Per bag and row, pool the $(N-1)P$ off-diagonal null weights, form the upper- q magnitude threshold $\tau_i^{(b)}$, and select edges with $|\hat{A}_{ij}^{(b)}| > \tau_i^{(b)}$; record q_Λ .
2. Compute selection frequencies π_{ij} and keep edges with $\pi_{ij} \geq \pi_{\text{thr}}$ (or use the `target_false_edges`-derived π_{thr}).
3. For surviving edges, compute the bagged mean $\bar{A}_{ij}$, bootstrap SD $\widehat{\text{SD}}_{ij}$ (selection-conditional and all-bag), and effect size z_{ij} against the pooled row null.
4. Emit the vetted edge list $(i, j, \pi_{ij}, \bar{A}_{ij}, \widehat{\text{SD}}_{ij}, z_{ij})$ plus diagnostics.

Closing note. The method’s robustness comes from never trusting a single fit: connectivity is asserted only when it survives data resampling (bagging), exceeds a null built by the identical estimation machinery (scramble calibration), and recurs across bags (stability selection). Its two reported uncertainties — the null spread and the across-bag bootstrap SD — are deliberately kept distinct, and their known limitations under overlapping bags are stated rather than papered over, so that the resulting edge list can be read with appropriate confidence on real data where no ground truth is available to check it.